SISTEMAS OPERATIVOS: PRÁCTICA 2

Autores: María Pozo, Carmen Gómez

1. Descripción de los requisitos

a) Arranque del sistema y proceso Principal:

- El proceso Principal crea el número de procesos especificado en los argumentos, guarda sus PID en un fichero, y luego envía la señal SIGUSR1 a los procesos Votante cuando el sistema está listo.

```c
/*Arguments verification and assignation.*/
if (argc != 3) {
    perror("The arguments should be: <N_PROCS> <N_SECS>");
    exit(EXIT_FAILURE);
}
n_procs = atoi(argv[1]);
n_secs = atoi(argv[2]);
if (n_procs <= 0 || n>secs <= 0) {
    perror("The arguments should be positive integers");
    exit(EXIT_FAILURE);
}
```

```c
/*Open INFO_SISTEMA file and write n_procs in it.*/
procFile = open(INFO_SISTEMA, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (procFile == -1) {
    perror("Error abriendo archivo");
    exit(EXIT_FAILURE);
}
snprintf(buffer, sizeof(buffer), "%d\n", n_procs);
write(procFile, buffer, strlen(buffer));

/*Allocate memory for pidsVoters.*/
pidsVoters = (int*) malloc(n_procs*sizeof(pidsVoters[0]));
if (!pidsVoters) {
    perror("Error alocando memoria");
    close(procFile);
    exit(EXIT_FAILURE);
}
```

```c
/*Create the voters processes.*/
for (i = 0; i < n_procs; i++) {
    pidVoter = fork();

    if (pidVoter == -1) {
        /*Control error.*/
        perror("Error creando un proceso Votante");
        close(procFile);
        free(pidsVoters);
        exit(EXIT_FAILURE);
    } else if (pidVoter == 0) {
        /*Voter process.*/
        signal(SIGINT, SIG_IGN);
        voter();
    } else {
        /*Principal process.*/
        pidsVoters[i] = pidVoter;
        snprintf(buffer, sizeof(buffer), "%d\n", pidsVoters[i]);
        write(procFile, buffer, strlen(buffer));
    }
}
```

```c
/*Send SIGUSR1 to all voter processes.*/
for (i = 0; i < n_procs; i++) {
    kill(pidsVoters[i], SIGUSR1);
}
```

- El proceso Principal también maneja señales de interrupción SIGINT, enviando la señal SIGTERM a los procesos Votante para terminar el sistema correctamente.

```c
/**
 * SIGINT signal handler.
 * Terminates all voter processes and frees memory.
 */
void handle_siginit(int sig) {
    /*Suppress unused parameter warning.*/
    (void) sig;
    int i;

    /*Send SIGTERM to all voter processes.*/
    for (i = 0; i < n_procs; i++) {
        kill(pidsVoters[i], SIGTERM);
    }

    free(pidsVoters);
    printf("Finishing by signal\n");
    exit(EXIT_SUCCESS);
}
```

b) Procesos Votante:

- Los procesos Votante esperan la señal SIGUSR1 para comenzar. Una vez que reciben esta señal, pueden intentar convertirse en el candidato y enviar la señal SIGUSR2 a los demás votantes para que registren su voto.

```c
while (1) {
    /*Suspend the process until a signal is received.*/
    sigsuspend(&old_set);
}
```

```c
read_systemInfo();

/*Lock the candidate mutex to start the voting process.*/
pthread_mutex_lock(&semCandidate);

/*Send SIGUSR2 to other voter processes to start voting.*/
for (i = 0; i < n_procsV; i++) {
    if (pidsVotersV[i] != getpid()) {
        kill(pidsVotersV[i], SIGUSR2);
    }
}
```

- Los votantes generan un voto aleatorio y lo registran en un fichero compartido.

```c
void handle_sigusr2(int sig) {
    /*Suppress unused parameter warning.*/
    (void) sig;

    /*Randomly vote "yes" (1) or "no" (0).*/
    int vote = rand() % 2;

    /*Lock the file mutex to safely write the vote to the file.*/
    pthread_mutex_lock(&semFile);
    int fd = open(VOTACION, O_WRONLY | O_APPEND | O_CREAT, 0644);
    if (fd == -1) {
        perror("Error abriendo archivo");
        free(pidsVotersV);
        exit(EXIT_FAILURE);
    }

    char buffer[4];
    snprintf(buffer, sizeof(buffer), "%d\n", vote);
    write(fd, buffer, strlen(buffer));
    fsync(fd);
    close(fd);

    /*Unlock the file mutex.*/
    pthread_mutex_unlock(&semFile);
}
```

c) Protección de zonas críticas:

- Se bloquean las señales y se realizan las esperas no activas con sigsuspend() para garantizar un sistema robusto.

```c
/*Block the signals to avoid race conditions while setting up handlers.*/
sigset_t set, old_set;
sigemptyset(&set);
sigaddset(&set, SIGUSR1);
sigaddset(&set, SIGTERM);
sigaddset(&set, SIGUSR2);
sigprocmask(SIG_BLOCK, &set, &old_set);
```

```c
while (1) {
    /*Suspend the process until a signal is received.*/
    sigsuspend(&old_set);
}
```

d) Temporización:

- El proceso Principal utiliza alarm() para asegurarse de que el sistema se termine después del número máximo de segundos introducido como argumento, enviando una señal SIGTERM a los procesos Votante cuando el tiempo ha expirado.

```
/*Set an alarm to trigger after n_secs.*/
alarm(n_secs);
```

```
/**
 * SIGALRM signal handler.
 * Terminates all voter processes and wait for their termination.
 */
void handle_sigalrm(int sig) {
    /*Suppress unused parameter warning.*/
    (void) sig;
    int i;

    /*Send SIGTERM to all voter processes.*/
    for (i = 0; i < n_procs; i++) {
        kill(pidsVoters[i], SIGTERM);
    }

    /*Wait for all voter processes to finish.*/
    for (i = 0; i < n_procs; i++) {
        waitpid(pidsVoters[i], NULL, 0);
    }

    free(pidsVoters);
    printf("Finishing by alarm\n");
    exit(EXIT_SUCCESS);
}
```

e) Análisis de la ejecución:

- **¿Se produce algún problema de concurrencia en el sistema descrito?**

Sí, en un sistema como el descrito pueden ocurrir problemas de concurrencia si no se protegen las zonas críticas; en este caso la elección del proceso Candidato y el acceso a los distintos archivos: el archivo que guarda la información del sistema (el número de procesos y los PID de los mismos) y el archivo que guarda las votaciones de cada proceso Candidato.

En principio, en el archivo de la información del sistema solo debería escribir el proceso Principal, por lo que no debería generar problemas de concurrencia.

Sin embargo, en el archivo de las votaciones escriben todos los procesos Votante, por lo que se debe controlar el acceso a este archivo para tener unos resultados correctos.

Los procesos Votante intentan convertirse en el proceso Candidato, pero sin sincronización varios procesos pueden convertirse a la vez en candidato.

- **¿Es sencillo, o siquiera factible, organizar un sistema de este tipo usando únicamente señales?**

No es sencillo y de hecho, tampoco es factible organizar un sistema de este tipo usando únicamente señales, pues las señales se utilizan para la comunicación entre procesos: la señal SIGUSR1 indica que el sistema está listo, SIGUSR2 indica que el candidato ha sido elegido y comienza la votación, y SIGTERM indica que los procesos deben terminar. No obstante, las señales no controlan la sincronización entre procesos.

Por lo tanto, aunque es posible usar señales en sistemas simples, un sistema más complejo como este necesita mecanismos de sincronización más robustos (como los semáforos o mutexes) que las señales por sí solas no pueden proporcionar.

f) Sincronización con semáforos:

```
sem_t *semFile, *semCandidate;
```

```
sem_t *semFile2, *semCandidate2;
```

```
sem_close(semFile);
sem_close(semCandidate);
sem_unlink(SEM_FILE);
sem_unlink(SEM_CANDIDATE);
```

- Usar semáforos para evitar problemas de concurrencia en la elección del proceso Candidato.

```
/*Create the file to chose the candidate*/
sem_unlink(SEM_CANDIDATE);
semCandidate = sem_open(SEM_CANDIDATE, O_CREAT, S_IRUSR | S_IWUSR, 1);
if (semCandidate == SEM_FAILED) {
    perror("Error inicializando semáforo de candidato");
    sem_close(semFile);
    sem_unlink(SEM_FILE);
    close(procFile);
    exit(EXIT_FAILURE);
}
```

```
/*Open the file to chose the candidate*/
semCandidate2=sem_open(SEM_CANDIDATE,0);
if (semCandidate2==SEM_FAILED) {
    perror("Error inicializando semáforo");
    exit(EXIT_FAILURE);
}
```

```
sem_wait(semCandidate2);
```

```
sem_post(semCandidate2);
```

- Usar semáforos para evitar problemas de concurrencia en los accesos al fichero.

```c
/*Create the file for the voters to vote*/
sem_unlink(SEM_FILE);
semFile = sem_open(SEM_FILE, O_CREAT, S_IRUSR | S_IWUSR, 1);
if (semFile == SEM_FAILED) {
    perror("Error inicializando semáforo de archivo");
    close(procFile);
    exit(EXIT_FAILURE);
}
```

```c
/*Open the file for the voters to vote*/
semFile2= sem_open(SEM_FILE, 0);
if (semFile2==SEM_FAILED) {
    perror("Error inicializando semáforo");
    exit(EXIT_FAILURE);
}
```

```c
vote = rand() % 2;
sem_wait(semFile2);
```

```c
sem_post(semFile2);
```

```c
sem_wait(semFile2);
fd = open(VOTACION, O_RDONLY);
```

```c
close(fd);
sem_post(semFile2);
```

2. Pruebas realizadas

```
maria@Conputas:/mnt/c/Users/yoyoy/OneDrive/Escritorio/soper-1/P2$ make
gcc -Wall -Wextra   -c -o principal.o principal.c
gcc -Wall -Wextra   -c -o voter.o voter.c
gcc -Wall -Wextra -o voting principal.o voter.o
```

- Prueba de argumentos no válidos: Se ha comprobado que el programa actúa correctamente cuando los argumentos introducidos en la ejecución no son válidos.

```
maria@Conputas:/mnt/c/Users/yoyoy/OneDrive/Escritorio/soper-1/P2$ ./voting
The arguments should be: <N_PROCS> <N_SECS>: Success
```

```
maria@Conputas:/mnt/c/Users/yoyoy/OneDrive/Escritorio/soper-1/P2$ ./voting -5 10
The arguments should be positive integers: Success
```

```
maria@Conputas:/mnt/c/Users/yoyoy/OneDrive/Escritorio/soper-1/P2$ ./voting 5 -10
The arguments should be positive integers: Success
```

```
maria@Conputas:/mnt/c/Users/yoyoy/OneDrive/Escritorio/soper-1/P2$ ./voting 5 0
The arguments should be positive integers: Success
```

```
maria@Conputas:/mnt/c/Users/yoyoy/OneDrive/Escritorio/soper-1/P2$ ./voting 0 10
The arguments should be positive integers: Success
```

```
maria@Conputas:/mnt/c/Users/yoyoy/OneDrive/Escritorio/soper-1/P2$ ./voting 5 10 15
The arguments should be: <N_PROCS> <N_SECS>: Success
```

- Prueba de creación de procesos votantes: Se ha verificado que el programa crea correctamente los procesos votantes al ejecutar el programa con diferentes valores de n_procs.

```
maria@Conputas:/mnt/c/Users/yoyoy/OneDrive/Escritorio/soper-1/P2$ ./voting 3 1
Finishing by alarm
```

ficheroProcesos.txt
```
3
11496
11497
11498
```

```
maria@Conputas:/mnt/c/Users/yoyoy/OneDrive/Escritorio/soper-1/P2$ ./voting 10 1
Finishing by alarm
```

ficheroProcesos.txt
```
10
11674
11676
11677
11678
11679
11680
11681
11682
11683
11684
```

```
maria@Conputas:/mnt/c/Users/yoyoy/OneDrive/Escritorio/soper-1/P2$ ./voting 20 1
Finishing by alarm
```

ficheroProcesos.txt
```
20
11810
11811
11812
11813
11814
11815
11816
11817
11818
11819
11820
11821
11822
11823
11824
11825
11826
11827
11828
11829
```

- Prueba de señales y manejo de interrupciones: Se ha comprobado el manejo adecuado de señales SIGINT y SIGALRM, asegurando que el sistema termine correctamente en estos casos y que los recursos se liberen.

```
maria@Conputas:/mnt/c/Users/yoyoy/OneDrive/Escritorio/soper-1/P2$ ./voting 3 5
^CFinishing by signal
```

```
maria@Conputas:/mnt/c/Users/yoyoy/OneDrive/Escritorio/soper-1/P2$ ./voting 3 5
Finishing by alarm
```

3. Funcionalidades adicionales

- **Manejo de señales:** El programa maneja varias señales de manera efectiva (SIGINT, SIGALRM, SIGUSR1, SIGUSR2, SIGTERM), lo que permite un control detallado del ciclo de vida de los procesos.
- **Uso de semáforos:** Se utilizan semáforos para proteger el acceso a recursos compartidos, como el fichero de votación y la elección de candidatos, lo que garantiza la integridad de los datos y evita condiciones de carrera.
- **Votación iterativa:** El sistema permite múltiples rondas de votación, con el candidato enviando señales SIGUSR1 a los demás procesos Votante después de cada ronda.
- **Almacenamiento de PID:** El programa guarda los PID de los procesos Votante en un fichero, lo que permite al proceso Principal saber exactamente qué procesos existen y enviarles señales según sea necesario.
- **Esperas no activas:** El programa implementa esperas no activas utilizando sigsuspend() y usleep(), lo que reduce el uso de CPU y permite que los procesos Votante permanezcan bloqueados de manera eficiente mientras esperan señales.